

软件分析与设计

第4讲 用例建模

重庆邮电大学软件工程学院

4.4 再论用例

识别参与者

- 如何识别参与者（即系统参与者）
 - 关键词：边界
 - 参与者：在系统之外，透过系统边界与系统进行有意义交互的任何事物



4.4 再论用例

识别参与者

- 参与者要点
 - 系统外
 - 参与者不是系统的一部分，处于系统的外部
 - 系统边界
 - 参与者透过系统边界直接与系统交互，参与者的确定代表系统边界的确定
 - 系统角色系统外
 - 参与者与使用系统的物理人和职务没有关系
 - 需要从参与系统的角色(作用)来寻找参与者

4.4 再论用例

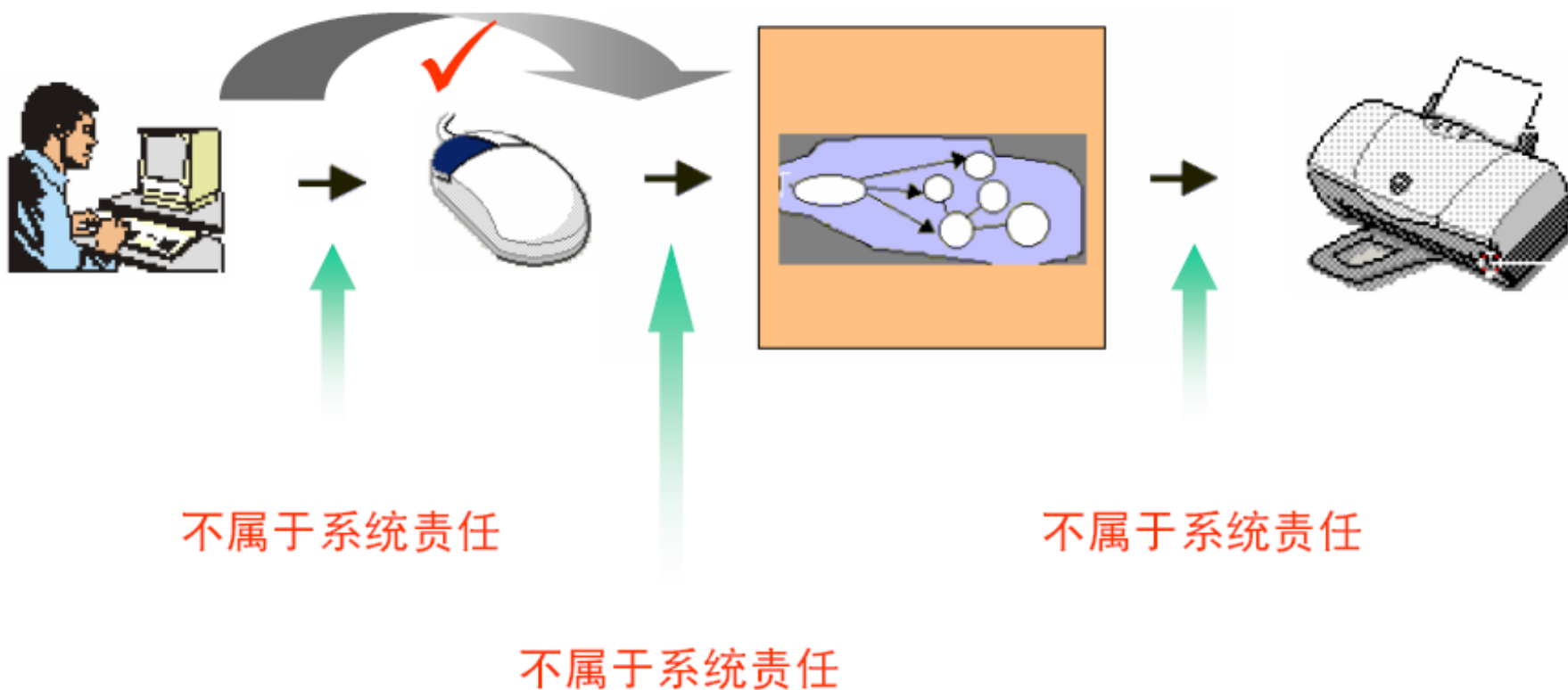
识别参与者

- 参与者要点
 - 与系统交互
 - 系统需要处理其交互过程，即系统职责
 - 任何事物
 - 人、外系统、外部因素、时间

4.4 再论用例

识别参与者

- 要点：与系统进行信息交互



4.4 再论用例

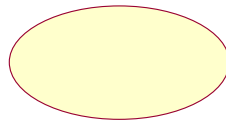
识别参与者

- 可以从以下要点来识别参与者
 - 系统在哪些部门使用
 - 谁向系统提供信息、使用和删除信息。
 - 谁与系统的需求有关联
 - 谁对系统进行维护
 - 与外部系统是否有关联
 - 时间参与者：一种习惯用法，用于激活那些系统定期的、自动执行的用例

4.4 再论用例

识别用例

- 如何识别用例
 - 关键词：价值
 - 简洁定义：参与者使用系统达到某个目标
 - 定义
 - 用例实例是系统执行的一系列动作，这些动作将生成特定参与者可观测的结果值
 - 一个用例定义一组用例实例（场景）



记住了，我是(系统)
用例

4.4 再论用例

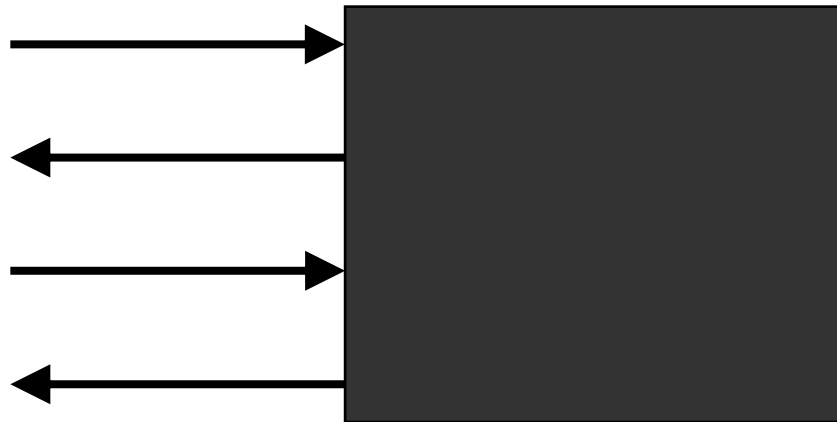
识别用例

- 用例要点
 - 可观测→用例止于系统边界
 - 结果值→用例是有意义的目标
 - 系统执行→结果值由系统生成
 - 由参与者观测→业务语言、用户观点

4.4 再论用例

识别用例

- 要点：用例止于系统边界

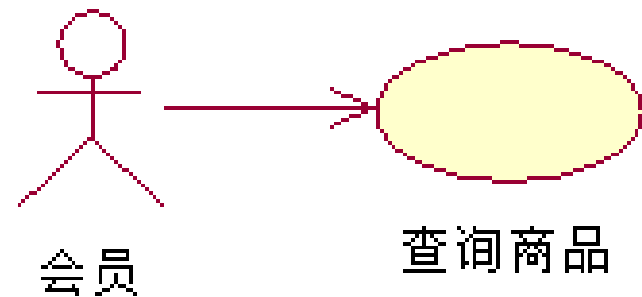
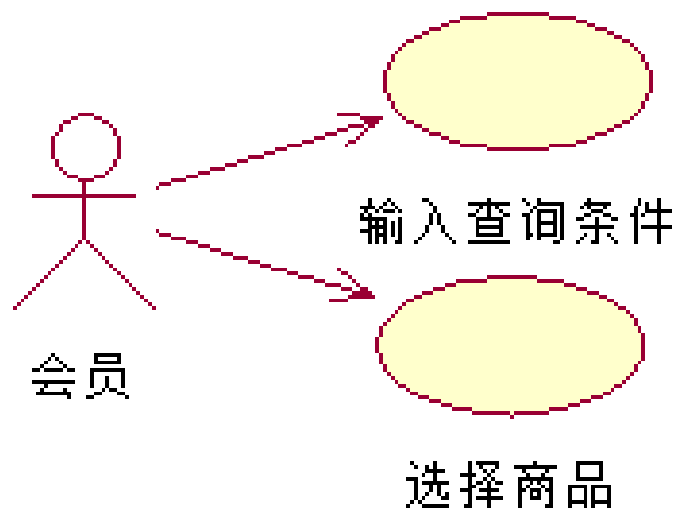


描述交互，而不是内在的系统活动

4.4 再论用例

识别用例

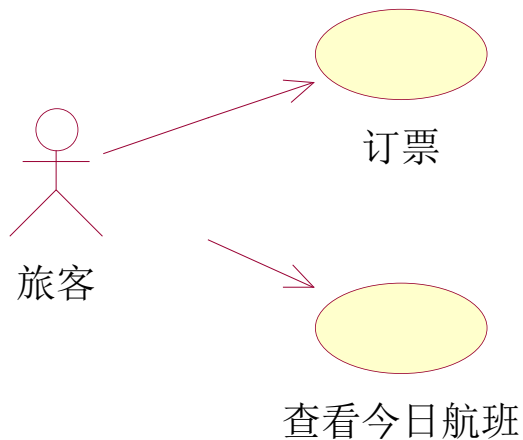
- 要点：有意义的目标



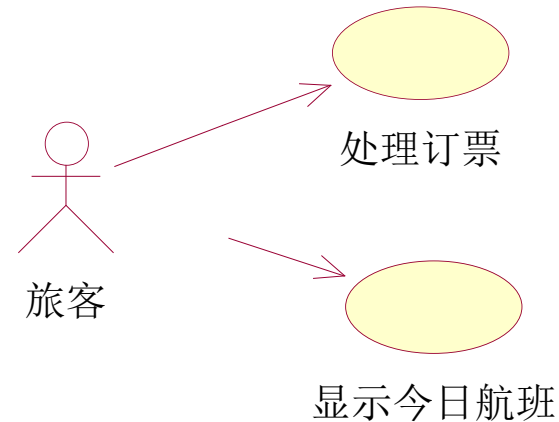
4.4 再论用例

识别用例

- 要点：用户观点而非系统观点



用户观点

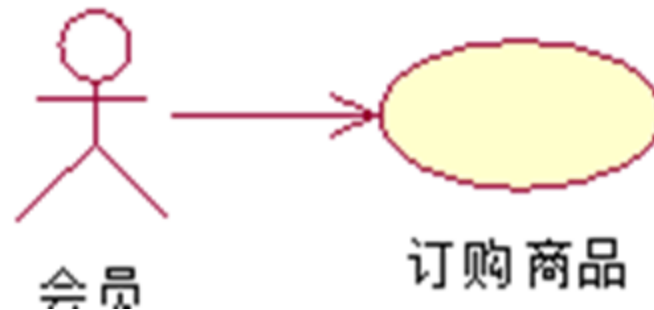


系统观点

4.4 再论用例

识别用例

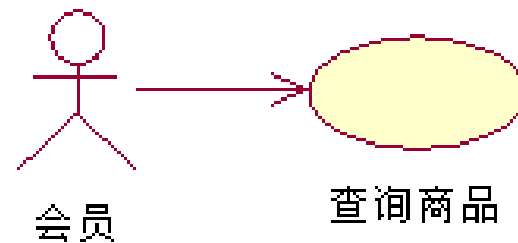
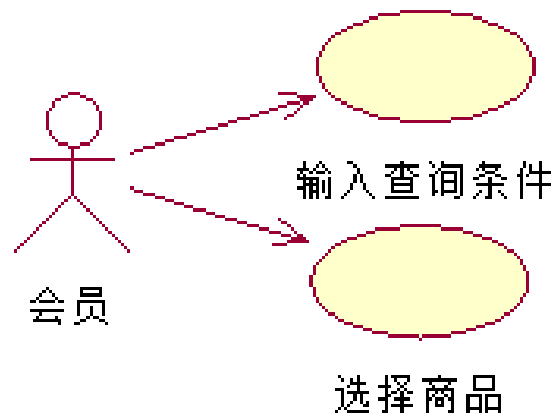
- 再次强调用例的命名
 - 站在参与者视角
 - （状语+）动词+（定语+）宾语



4.4 再论用例

用例粒度

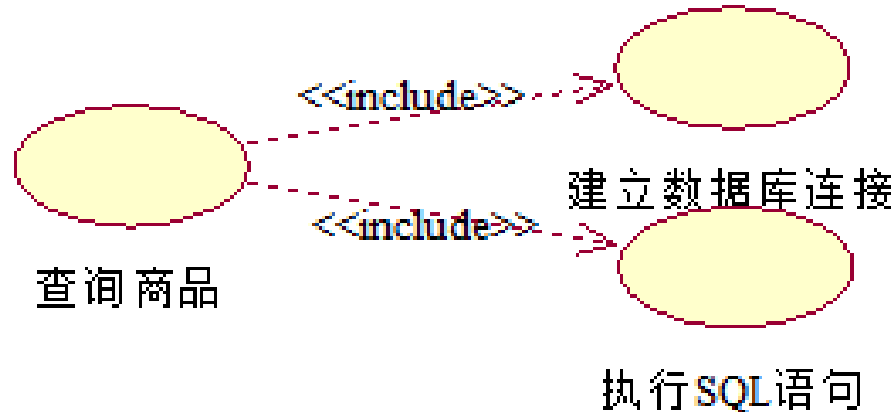
- 把步骤当用例



4.4 再论用例

用例粒度

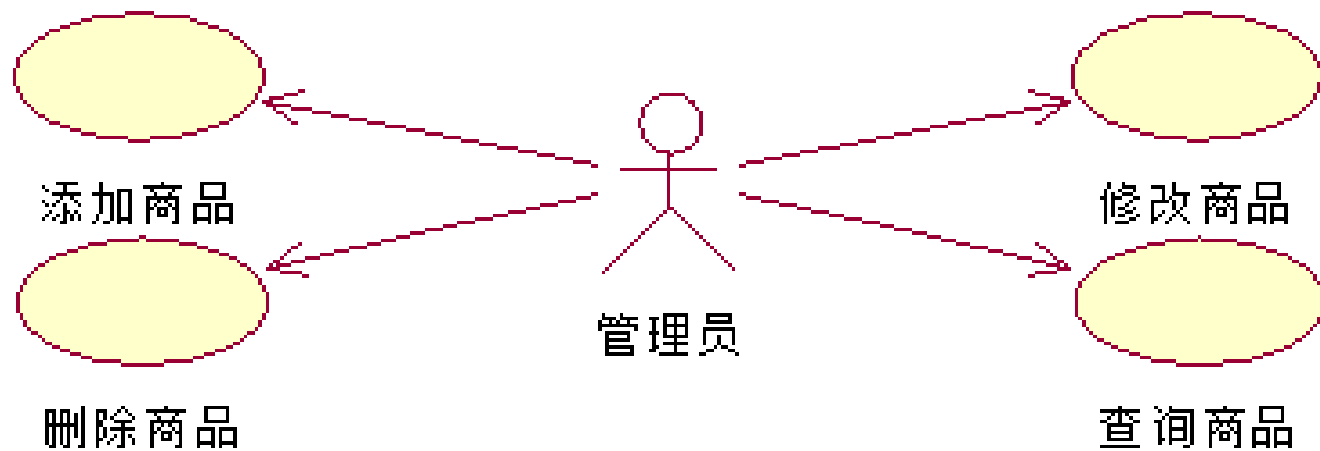
- 把系统活动当用例



4.4 再论用例

用例粒度

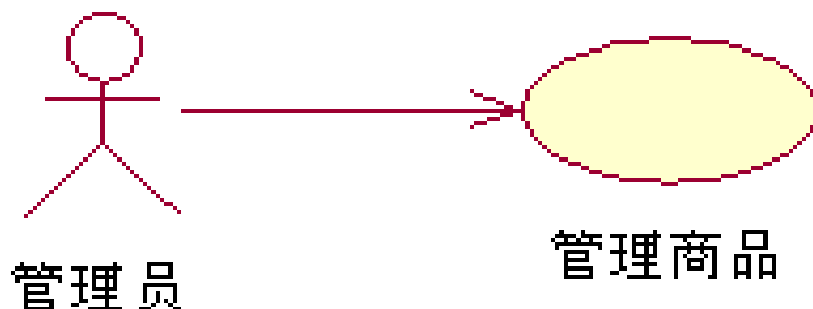
- 四轮马车**CRUD**:
 - 所有业务最终都会成为**CRUD**?
 - **CRUD**能为**Actor**提供价值?
 - **CRUD**掩盖业务，锐变成关系数据库的建模
 - “系统就是数据的增删改查”
 - 关心数据的存储和维护，反而忽略了用户的目的



4.4 再论用例

用例粒度

- 如果确实是**CRUD**
 - 如果**CRUD**不涉及复杂的交互，一个用例“管理××”即可
 - 不管是**C**、**R**、**U**、**D**，都是为了完成“管理”目标
 - 甚至很多种的基本数据管理都可以用一个用例表示

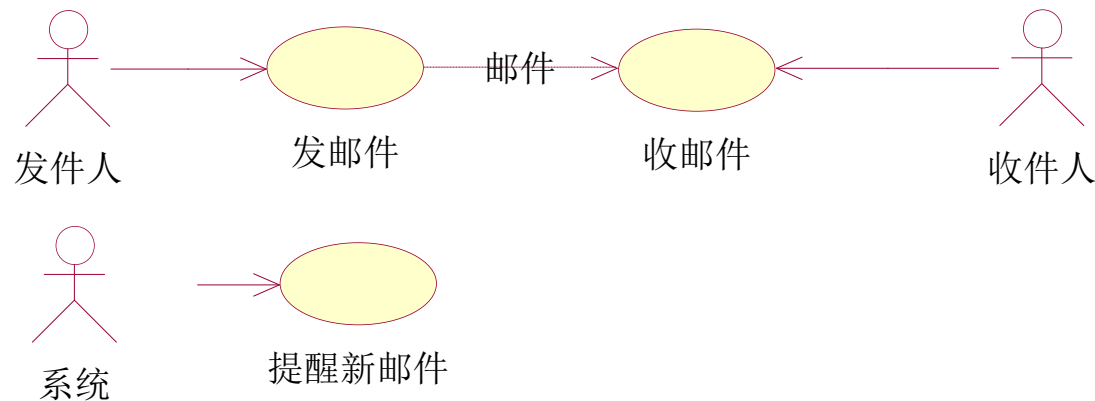


4.4 再论用例

识别用例

- 思考练习

- **Email**客户端（如：**outlook**），**A**在北京发邮件给上海的**B**，系统提醒**B**你有“新邮件”，**B**收邮件



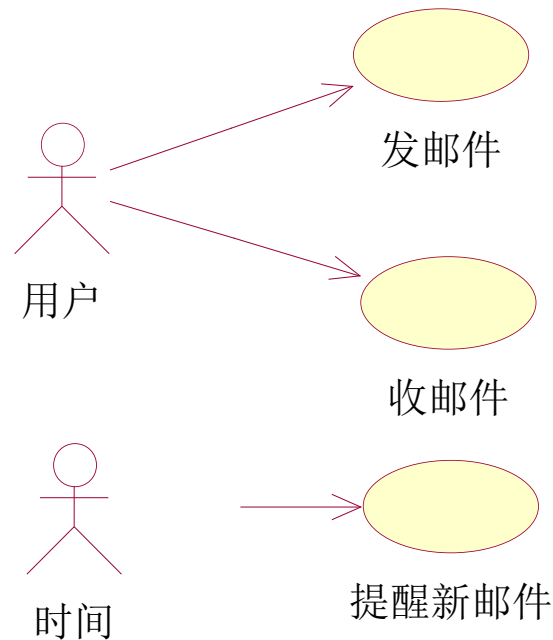
用例是一个完整的交互
用例之间没有顺序的关系

4.4 再论用例

识别用例

- 思考练习

- Email客户端（如：outlook），A在北京发邮件给上海的B，系统提醒B你有“新邮件”，B收邮件



4.6 用例规约

- 用例规约是需求的核心内容
 - 描述用例与外界交互的规格说明书
 - 用例图作为用例文档的索引图
 - 进一步的精度：有层次的文档
 - 文档中每一句话都有其价值

用例图是骨架，而用例文档则是其内在的肉

4.6 用例规约

用例规约组成

- 用例名、简要描述
- 参与者与涉众
- 相关用例
- 前置条件、后置条件
- 事件流
 - 基本路径
 - 备选路径
- 补充约束
 - 字段列表、业务规则
 - 非功能需求、设计约束

4.6 用例规约

用例规约组成

用例名	预订房间
简要描述	旅店的服务员通过该用例为顾客预订所需的房间
参与者	服务员
涉众	服务员：准确地完成预订过程 旅客：简单、快速地预订到所需的房间
相关用例	无
前置条件	服务员正确登录到该系统
后置条件	如果预订成功，则系统记录本次预订信息，相关的房间状态被正确标识

基本事件流

- (1) 用例起始于旅客现场需要预订房间
- (2) 服务员按照旅客的要求设定查询条件(D-1)来查询可预订的房间信息(D-2)
- (3) 系统显示所有可预订的房间列表(A-1)(B-1)
- (4) 服务员为旅客选定所需的房间(A-2)，并输入预订的时间和天数
- (5) 系统计算所需的总费用和预付订金金额(B-2)
- (6) 旅客现场用现金支付所需的订金(A-3)
- (7) 服务员将支付信息记录到系统中，并进行预订操作
- (8) 系统保存本次预订信息(D-3)，显示预订成功消息(A-4)
- (9) 系统打印预订收据后，用例结束

备选事件流

- A-* 服务员在保存预订信息前，随时都可以中止该用例
- (1) 系统提醒服务员当前所有操作都会被取消
 - (2) 服务员确认后，当前用例结束，也可选择取消，则继续后续操作
- A-1 没有找到满足可预订需求的房间
- (1) 系统显示没有找到所需的房间
 - (2) 服务员可以重新设定查询条件，也可选择结束该用例
- A-2 所有可预订房间中没有旅客满意的房间
- 服务员可以重新查询其他房间，也可选择结束该用例
- A-3 旅客没有足够的现金来支付订金
- 服务员可以重新为旅客选择其他房间，或修改预订时间，也可以选择结束该用例
- A-4 系统保存失败
- (1) 系统显示保存失败信息，并提醒服务员重新提交
 - (2) 服务员可以重新提交本次预订信息，也可以结束该用例

补充约束-数据需求

- D-* 房间的状态包括空闲、预订、占用、维修
- D-1 查询条件包括预订时间段、房间类型
- D-2 房间信息包括房间号、房间类型、价格、房间状态
- D-3 预订信息包括客户的基本信息(姓名、地址、联系电话、有效证件号)和本次预订情况(房间号、预订天数、预订时间、预订金额)

补充约束-业务规则

- B-1 每个房间同一天只允许一个人预订，可以根据房间的状态来决定该房间是否可被预订
- B-2 预订时按房间的总费用需要支付一定比例的预订金额，具体比例可以灵活设定，目前初步定为10%

补充约束-非功能需求

可支持性：目前只考虑旅客用现金当场支付的情况，但也要为其他支付方式预留接口

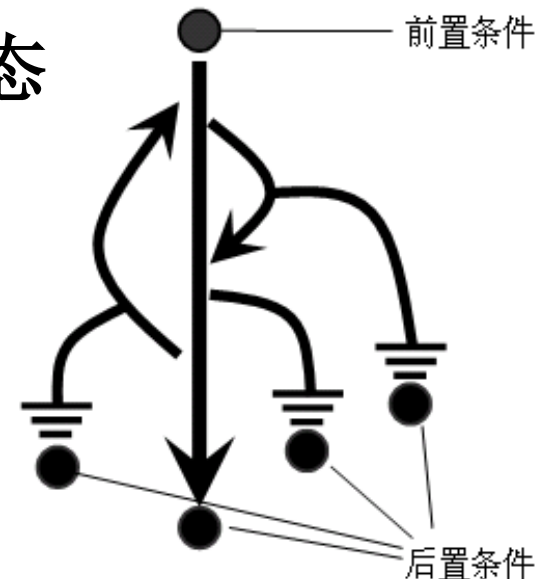
待解决问题

(暂无)

4.6 用例规约

前置条件和后置条件

- 前置条件约束在用例开始前系统的状态
 - 作为用例的入口限制，它阻止参与者触发该用例，直到满足所有条件
 - 说明在用例触发之前什么必须为真
- 后置条件约束用例执行后系统的状态
 - 用例执行后什么必须为真
 - 对于存在各种分支事件流的用户例，则可以指定多个后置条件

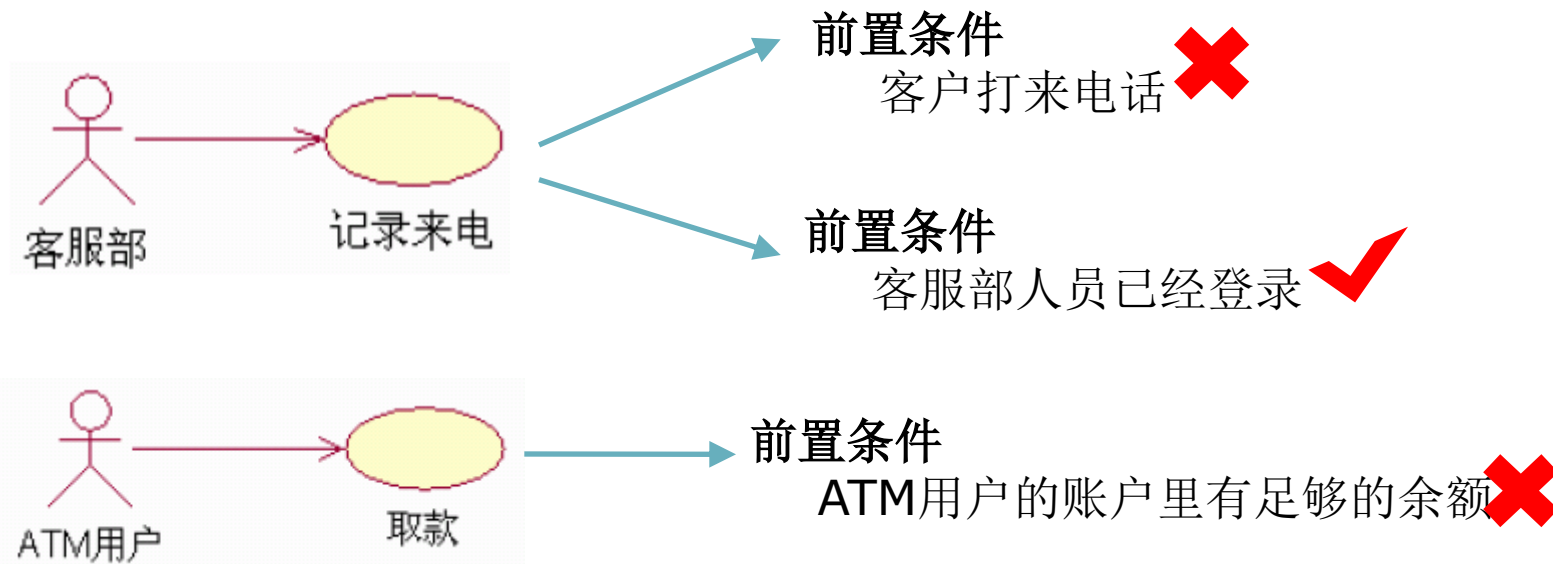


4.6 用例规约

前置条件和后置条件

- 定义前置、后置条件

- 只有在用例的使用者将这些条件视为附加价值是才使用
- 条件必须是系统可以感知的
- 前置条件必须是在用例执行前就可以感知



4.6 用例规约

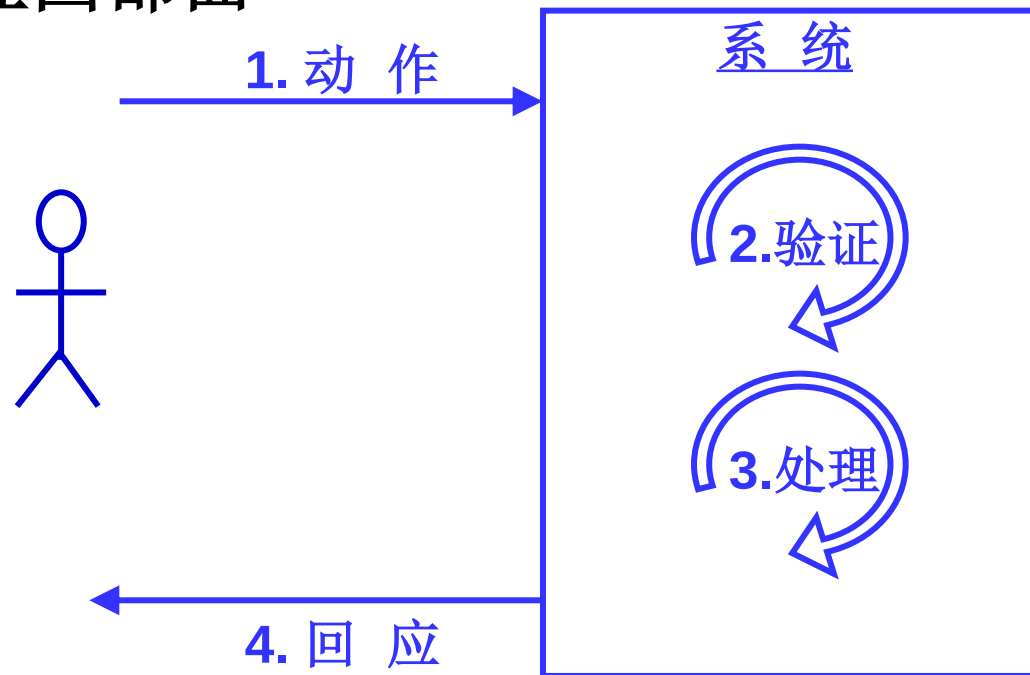
前置条件和后置条件

- 应用前置、后置条件
 - 前置和后置条件是用例的可选特征，如果系统状态对用例如何启动和如何终止不重要，则可以省略这些条件只有在用例的使用者将这些条件视为附加价值是才使用
 - 某些用例依赖于其他用例
 - 例如：“登录”和“管理系统”
 - 有助于识别漏掉的用例
 - 例如：“管理订单”却没有“登录”用例

4.6 用例规约

事件流

- 用例交互四部曲



重点写：1和4（可观测的、体现客户利益的文字）

4.6 用例规约

事件流

- 事件流描述要点

- 使用业务语言：使用用户平时所使用的语言进行描述
- 重点描述参与者与系统交互的信息
- 不使用“例如”、“等”不清晰的表达
- 不要过多的考虑界面细节
- 不要描述系统内部处理细节，要描述从系统外部所看到的活动
- 要明确描述用例的开始和结束
- 不仅需要描述基本事件流，还需要考虑备选事件流

4.6 用例规约

事件流

- 两种类型的事件流

- 基本事件流

- 用例的主路径、愉快路径（**Happy Path**）
 - 通常用来描述一个理想世界，即没有任何错误发生的情况
 - 复杂的基本流可以分解成多个子流

- 备选事件流

- 基本事件流中的分支或异常情况
 - 注意如何与基本流衔接

4.6 用例规约

补充约束

- 用例重点在于描述功能需求，而其它方面的补充约束：两种类型的事件流
 - 与特定用例相关的补充约束，作为该用例文档中一部分来描述
 - 一些全局性的补充约束，单独形成一份独立的文档，如“补充需求规约”文档
 - 补充约束包括：
 - 数据需求
 - 业务规则
 - 非功能需求
 - 设计约束

Thank You !